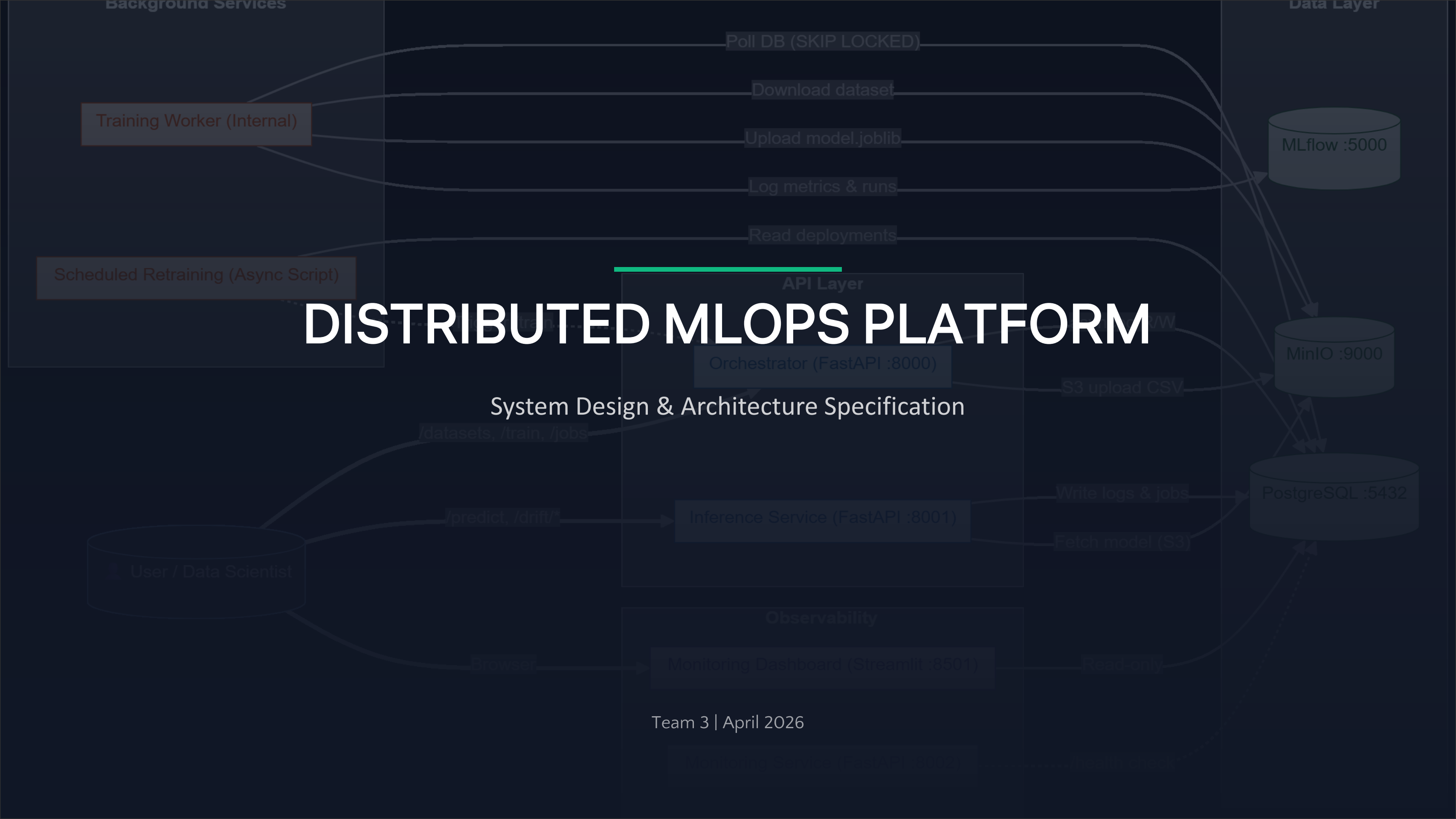


DISTRIBUTED MLOPS PLATFORM

System Design & Architecture Specification

Team 3 | April 2026



What We'll Cover

01

The Problem

Why ML models get stuck in notebooks and never reach production

02

System Architecture

How the platform is built and how components interact

03

Key Features

Training, inference, storage, and monitoring capabilities

04

Roadmap & Team

Where we're headed and who built this platform

01



The Problem

Why we built this platform and what it solves

From Notebook to Production

The Challenge

ML models often get stuck in experimental environments and never reach production.

The Pain Points

- Fractured data pipelines
- Lost model traceability
- Scalability limits (Python GIL)

Our Solution

A lightweight infrastructure managing the full ML lifecycle — from automated training to optimized inference.

Before vs. After

BEFORE

Manual model tracking in notebooks

No versioning or reproducibility

Hand-off gaps between data scientists and engineers

Deployments are fragile and error-prone

AFTER

Automated training pipeline with state tracking

Full model lineage via MLflow + MinIO

One platform for scientists and engineers

Resilient, observable, production-ready

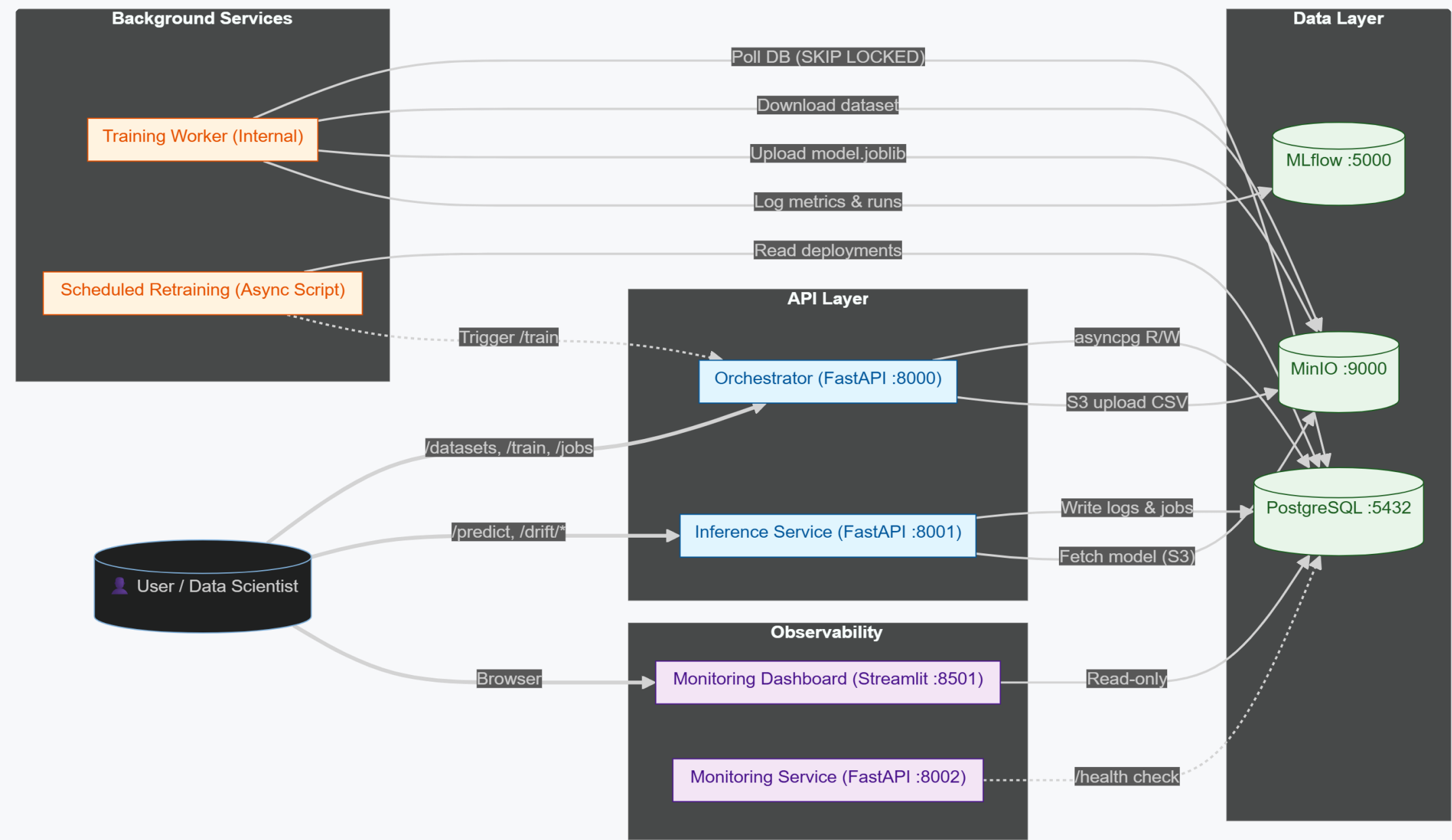
02

System Architecture

How the platform is built and how components interact
interact

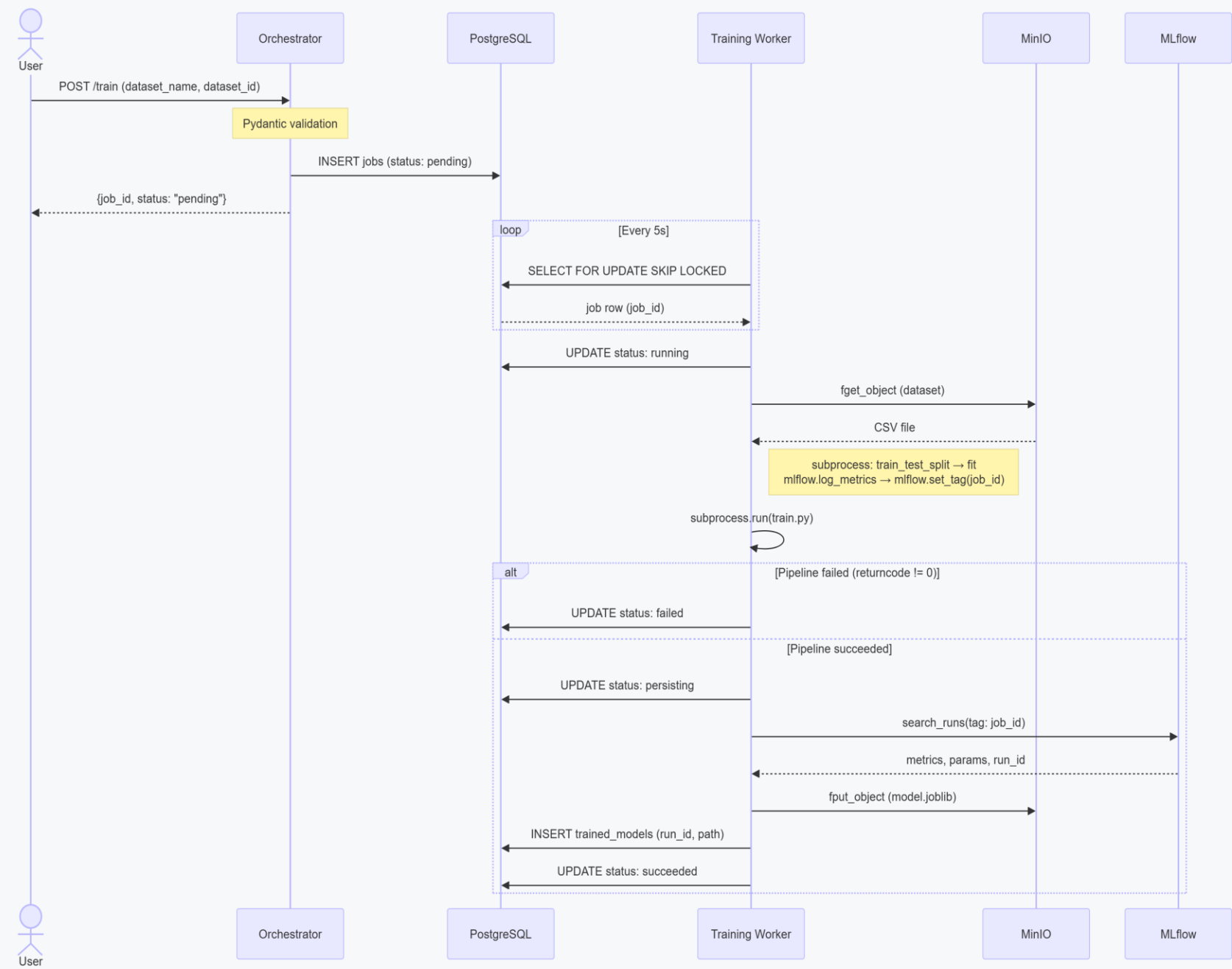
Three-Plane Architecture

9 containerized services running on Docker Compose



Model Lifecycle

From raw data to live inference in 7 steps



- 1. Upload dataset via API
- 2. Request training job
- 3. Worker polls & atomically locks job
- 4. Subprocess executes ML pipeline
- 5. Metrics logged to MLflow
- 6. Model persisted to MinIO
- 7. Job finalized as succeeded

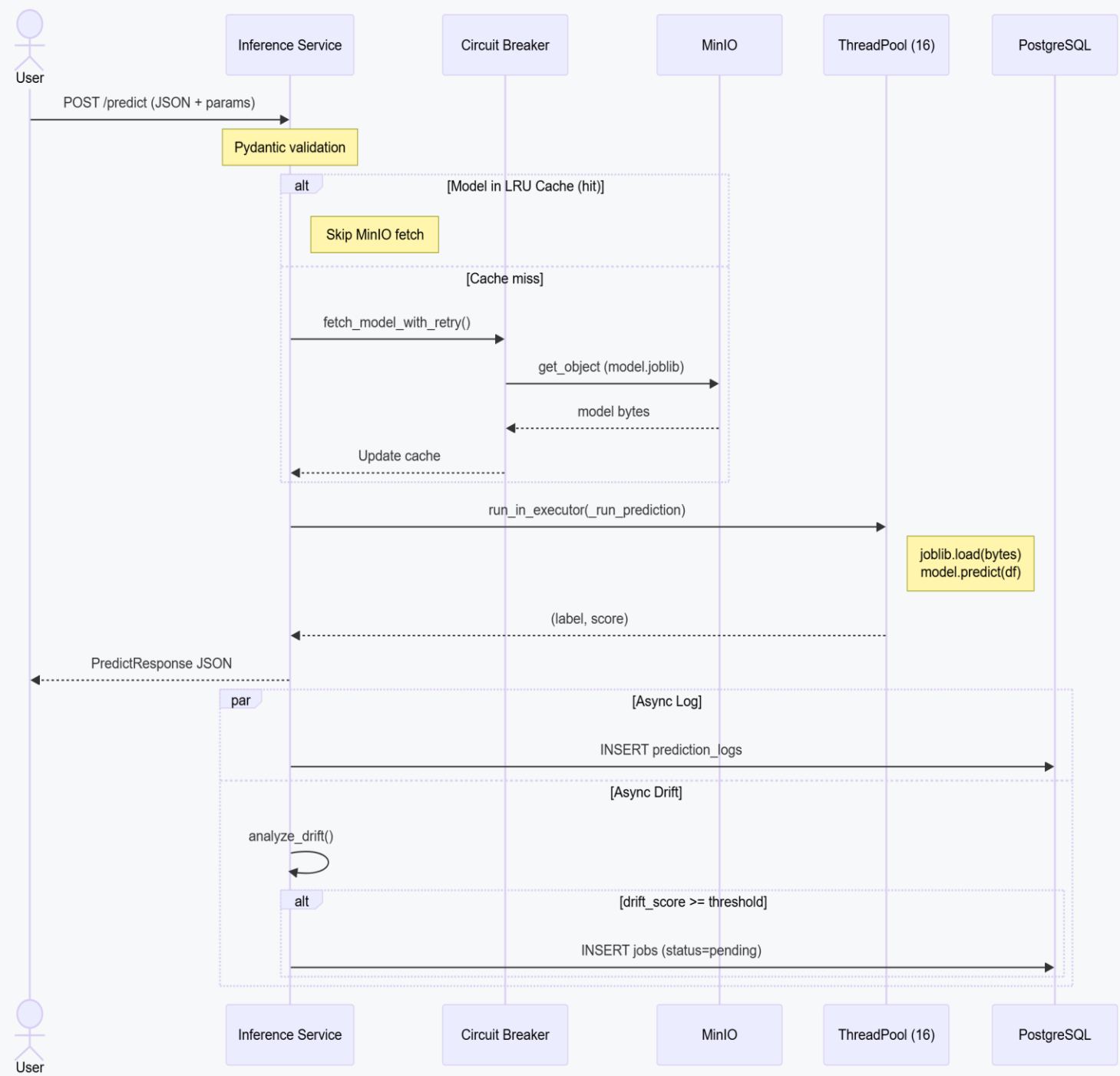
Resilience: If a worker crashes, stuck jobs auto-recover to pending state on next startup.

03

Key Features

Training, inference, storage, and monitoring capabilities
capabilities

Smart Training & Fast Inference



Training Worker

OS-level subprocess isolation prevents memory crashes

Scikit-Learn pipeline with MLflow experiment tracking

Auto-recovery for stuck jobs on worker restart



Inference Service

16-thread ThreadPool for concurrent predictions

In-process model caching + per-thread deserialization

Circuit Breaker for MinIO resilience

Storage & Monitoring



MinIO Storage

S3-compatible object storage with dedicated buckets for datasets and serialized models



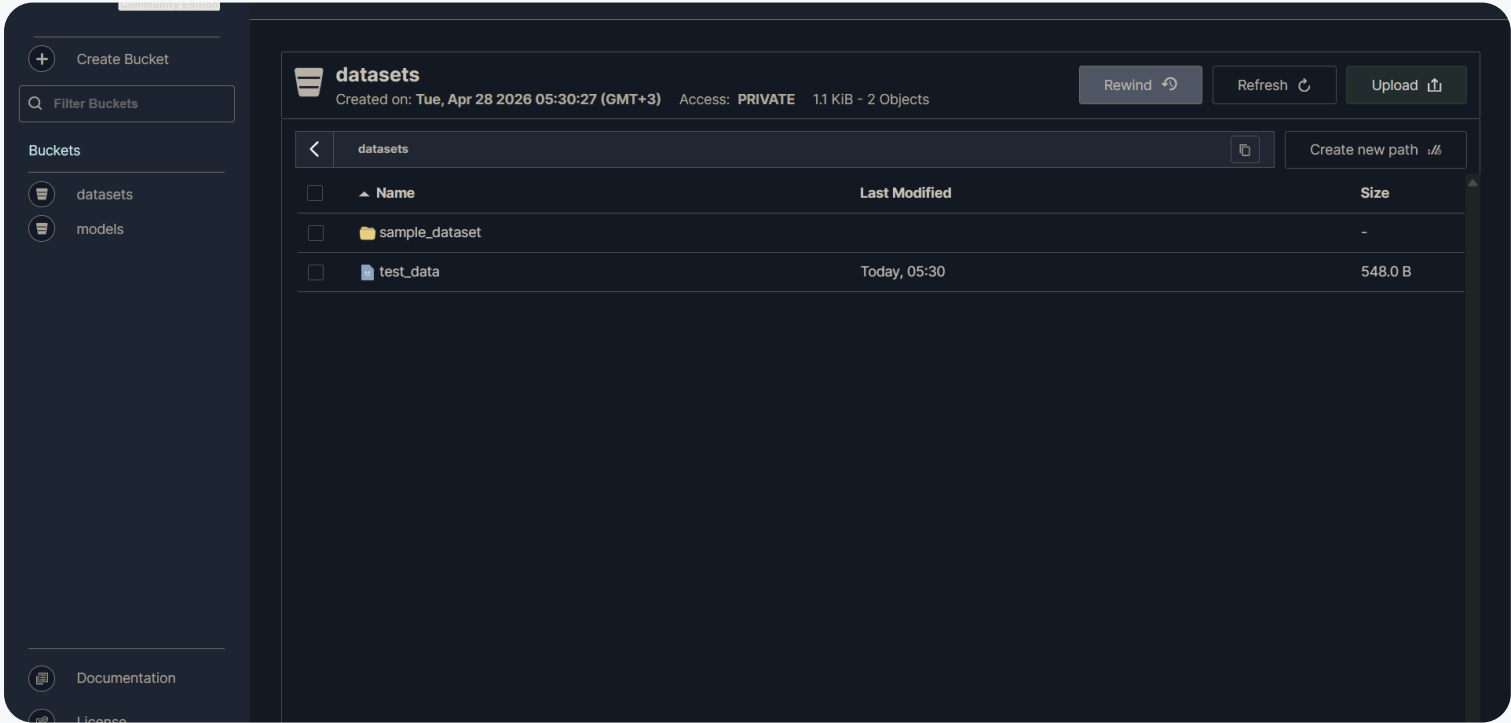
MLflow Tracking

Experiment registry logging Accuracy, Precision, Recall, F1 — all linked to job IDs

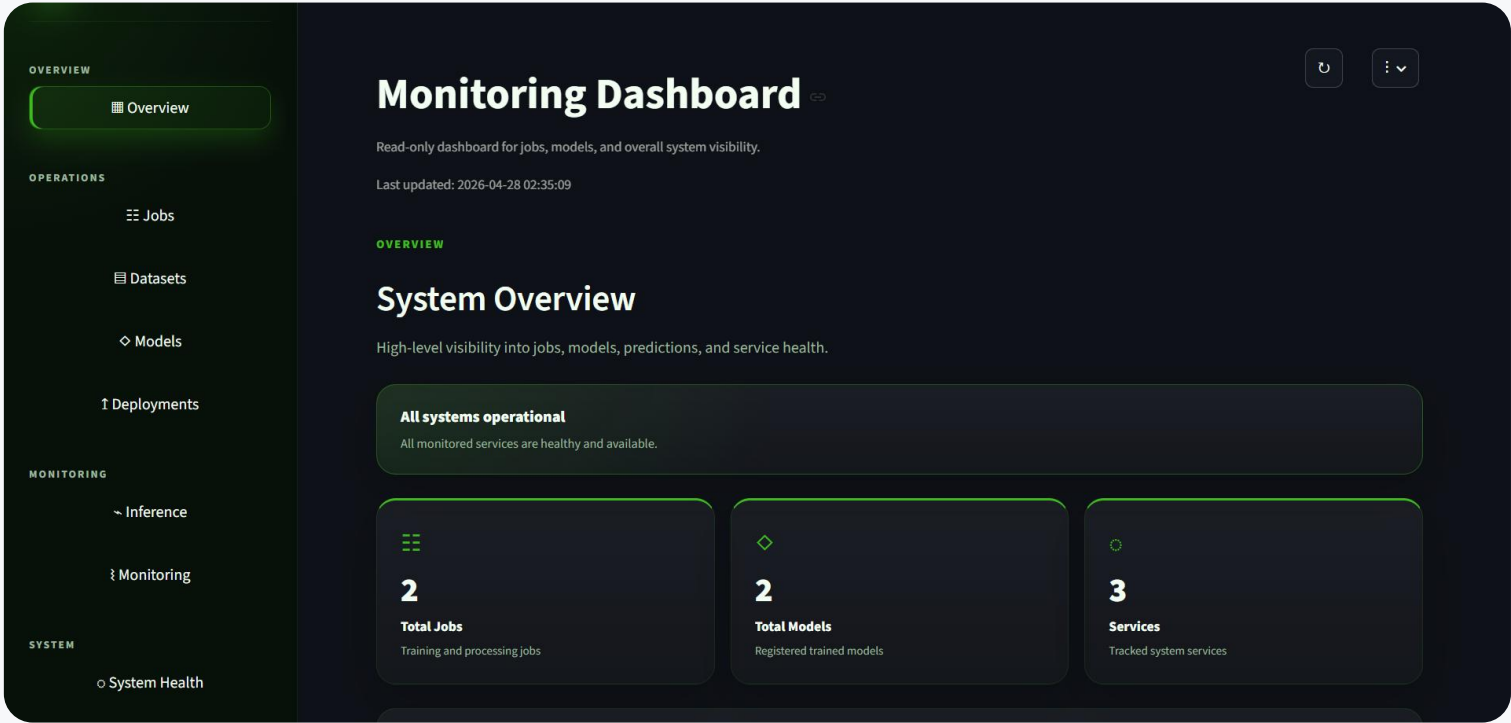


Streamlit Dashboard

8-page monitoring app: Overview, Jobs, Datasets, Models, Deployments, Inference, Health



MinIO Object Store



Dashboard System Overview

04

Roadmap & Team

Where we're headed and who built it

Meet the Team

8 engineers, 3 weeks, one platform



Andrei P.

Team Lead
CI/CD, Architecture



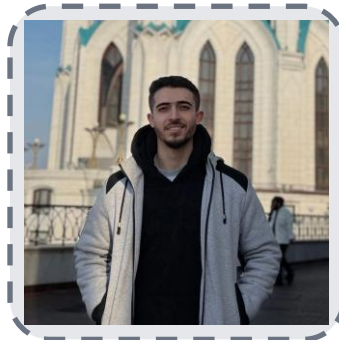
Arina S.

Project Manager
Coordination



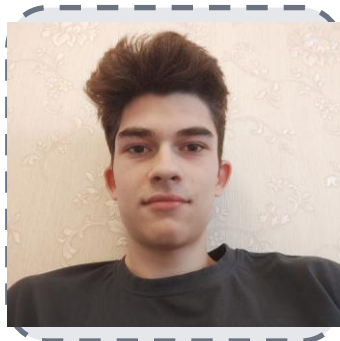
Elizaveta K.

Orchestrator
Database Design



Ahmed L.

Dashboard



Aleksei

Inference Service
Circuit Breaker



Danil K.

Training Worker
MLflow Integration



Daria G.

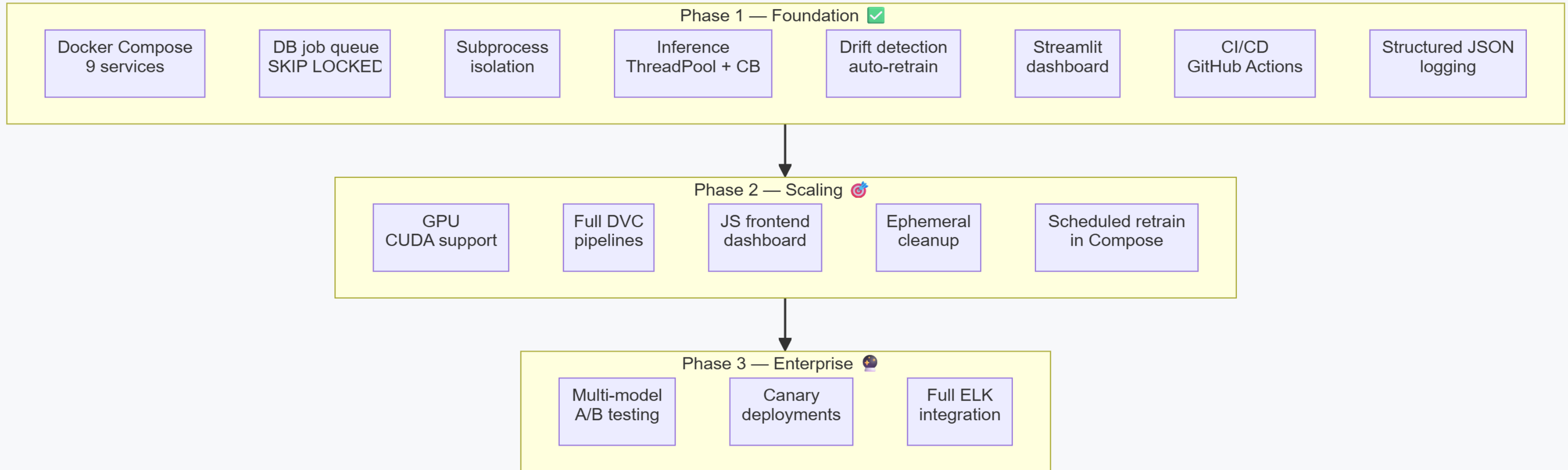
Training Pipelines
Dataset Registration



Julia K.

Reports
Presentations

Development Roadmap



Phase 1 — Foundation

Docker Compose, async jobs, inference, dashboard, CI/CD — complete

Phase 2 — Scaling

GPU support, DVC data lineage, JS frontend, ephemeral cleanup

Phase 3 — Enterprise

Multi-model A/B testing, canary deployments, full ELK integration

THANK YOU

Questions?

Team 3 | Distributed MLOps Platform